



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERIA

Escuela Profesional de Ingeniería de Sistemas

INFORME DE LABORATORIO
“Formulario Registro de Contactos”

Curso: Programación II

Docente: Ing. Breno Luque Zuñiga

Tarqui Ramos, Sebastián Emir Andree (2022075482)

Tacna – Perú
2026



INDICE

I. INFORMACIÓN GENERAL	3
- Objetivos:	3
- Equipos, materiales, programas y recursos utilizados:	3
II. MARCO TEORICO	3
III. PROCEDIMIENTO	4
IV. ANALISIS E INTERPRETACION DE RESULTADOS	9
V. CUESTIONARIO	10
CONCLUSIONES	10



INFORME DE LABORATORIO

Formulario Registro de Contactos

I. INFORMACIÓN GENERAL

- **Objetivos:**
 - Implementar un sistema de registro de contactos aplicando los principios de POO: encapsulamiento, clases y objetos.
 - Separar la lógica de negocio de la interfaz gráfica mediante una clase de servicio (ContactoService).
 - Usar colecciones dinámicas (List<T>) para almacenar y manipular contactos en memoria.
 - Aplicar validaciones de datos tanto en tiempo de escritura (KeyPress) como antes de guardar (ValidarCampos).
 - Manejar eventos de formulario y controles como TextBox, ComboBox, DataGridView y Button.

- **Equipos, materiales, programas y recursos utilizados:**
 - Computadora con Windows 10/11
 - Visual Studio 2022 Community Edition
 - .NET Framework / .NET 6+ (Windows Forms)
 - Lenguaje de programación: C#
 - Librería del sistema: System.Text.RegularExpressions, System.Collections.Generic, System.Linq
 - Imagen de referencia del formulario proporcionada por el docente.

II. MARCO TEORICO

Programación Orientada a Objetos (POO)

La POO es un paradigma de programación que organiza el software en torno a objetos, los cuales combinan datos (atributos) y comportamiento (métodos). Sus principios fundamentales son encapsulamiento, herencia, polimorfismo y abstracción.

Clases y Objetos

Una clase es una plantilla que define la estructura y comportamiento de un objeto. En este proyecto la clase Contacto encapsula los atributos ID, Nombre, Apellido, NumContacto, Dirección y Genero, con sus respectivas propiedades públicas.

Separación de responsabilidades

La clase ContactoService concentra toda la lógica de negocio: agregar, actualizar, eliminar, buscar y listar contactos. El formulario Form1 solo se ocupa de la interfaz y delega las operaciones al servicio. Esto facilita el mantenimiento y la reutilización del código.

List<T> - Colecciones dinámicas

List<T> es una colección genérica de C# que permite agregar, eliminar y recorrer elementos de forma dinámica sin necesidad de definir un tamaño fijo, a diferencia de los arreglos convencionales.

Validación de datos

Se aplican dos niveles de validación: (1) en tiempo real mediante el evento KeyPress, que impide ingresar caracteres inválidos directamente en el TextBox; (2) antes de guardar mediante expresiones regulares (Regex) que verifican el formato correcto de nombre, apellido y número telefónico.

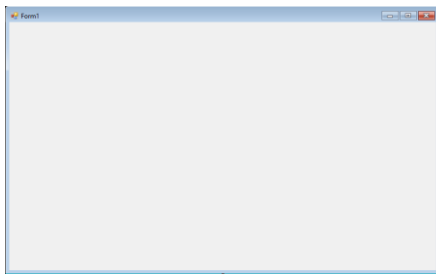
DataGridView

Control de Windows Forms que muestra datos en formato de tabla. Se configura con columnas explícitas vinculadas a las propiedades del objeto Contacto mediante DataPropertyName, y se actualiza asignando una nueva lista como DataSource.

III. PROCEDIMIENTO

1. Creación del proyecto

Se creó un proyecto Windows Forms App en Visual Studio 2022 con el nombre GestionContactos.



2. Clase Contacto

Se creó la clase Contacto.cs con las propiedades: ContactoID (int), Nombre, Apellido, NumContacto, Direccion y Genero (string). Se implementó un constructor con parámetros y otro vacío.

```
9 public class Contacto
10 {
11     8 referencias
12     public int ContactoID { get; set; }
13     5 referencias
14     public string Nombre { get; set; }
15     5 referencias
16     public string Apellido { get; set; }
17     5 referencias
18     public string NumContacto { get; set; }
19     3 referencias
20     public string Direccion { get; set; }
21     3 referencias
22     public string Genero { get; set; }
23
24     0 referencias
25     public Contacto() { }
26
27     1 referencia
28     public Contacto(int id, string nombre, string apellido, string numContacto, string direccion, string genero)
29     {
30         ContactoID = id;
31         Nombre = nombre;
32         Apellido = apellido;
33         NumContacto = numContacto;
34         Direccion = direccion;
35         Genero = genero;
36     }
37
38     0 referencias
39     public override string ToString()
40     {
41         return $"{Nombre} {Apellido} - {NumContacto}";
42     }
43 }
```

3. Clase ContactoService

Se implemento ContactoService.cs con una lista privada List<Contacto> y un contador _nextId para autogenerar IDs. Los métodos implementados fueron: ObtenerTodos(), Agregar(), Actualizar(), Eliminar(), Buscar() y ObtenerPorId().

```
9 public class ContactoService
10 {
11     private List<Contacto> _contactos = new List<Contacto>();
12     private int _nextId = 1;
13
14     1 referencia
15     public List<Contacto> ObtenerTodos() => new List<Contacto>(_contactos);
16
17     1 referencia
18     public void Agregar(Contacto c)
19     {
20         if (_contactos.Any(x => x.ContactoID == c.ContactoID))
21             throw new Exception("Ya existe un contacto con ese ID.");
22
23         c.ContactoID = _nextId++;
24         _contactos.Add(c);
25     }
26
27     1 referencia
28     public void Actualizar(Contacto c)
29     {
30         var existente = _contactos.FirstOrDefault(x => x.ContactoID == c.ContactoID);
31         if (existente == null)
32             throw new Exception("Contacto no encontrado.");
33
34         existente.Nombre = c.Nombre;
35         existente.Apellido = c.Apellido;
36         existente.NumContacto = c.NumContacto;
37         existente.Direccion = c.Direccion;
38         existente.Genero = c.Genero;
39     }
40
41     public void Eliminar(int id)
42     {
43         var c = _contactos.FirstOrDefault(x => x.ContactoID == id);
44         if (c == null) throw new Exception("Contacto no encontrado.");
45         _contactos.Remove(c);
46     }
47
48     1 referencia
49     public List<Contacto> Buscar(string texto)
50     {
51         texto = texto.ToLower();
52         return _contactos.Where(c =>
53             c.Nombre.ToLower().Contains(texto) ||
54             c.Apellido.ToLower().Contains(texto) ||
55             c.NumContacto.Contains(texto)
56         ).ToList();
57     }
58
59     0 referencias
60     public Contacto ObtenerPorId(int id) =>
61         _contactos.FirstOrDefault(x => x.ContactoID == id);
62 }
```

4. Diseño del formulario

Se diseño Form1 mediante el Designer de Visual Studio agregando los controles según la imagen de referencia. A continuación, se detallan todos los controles utilizados:



Control	Nombre	Descripción
TextBox	txtID	Muestra el ID autogenerado del contacto seleccionado (solo lectura referencial)
TextBox	txtNombre	Ingreso del nombre del contacto (solo letras y espacios)
TextBox	txtApellido	Ingreso del apellido del contacto (solo letras y espacios)
TextBox	txtNumContacto	Ingreso del número telefónico (solo dígitos)
TextBox	txtDireccion	Ingreso de la dirección (multilínea)
ComboBox	cmbGenero	Selección del género: Masculino, Femenino u Otro
TextBox	txtBuscar	Campo de búsqueda en tiempo real por nombre, apellido o numero
DataGridView	dgvContactos	Visualización tabular de todos los contactos registrados
Button	btnAgregar	Agrega un nuevo contacto a la lista (verde)



Button	btnActualizar	Actualiza los datos del contacto seleccionado (azul)
Button	btnBorrar	Elimina el contacto seleccionado previa confirmación (rojo)
Button	btnClaro	Limpia el formulario y recarga el listado completo (naranja)

5. Lógica del formulario (Form1.cs)

En el constructor se suscribieron todos los eventos. Se implementaron los métodos:

ConfigurarDGV() para definir columnas del grid, CargarDatos() para refrescar la lista,

ValidarCampos() con Regex para validaciones, ContactoDesdeFormulario() para construir un

objeto desde los controles, y LimpiarFormulario() para limpiar la vista.

```
14 public partial class Form1 : Form
15 {
16     private readonly ContactoService _service = new ContactoService();
17     1 referencia
18     public Form1()
19     {
20         InitializeComponent();
21         ConfigurarDGV();
22         CargarDatos();
23     }
24     1 referencia
25     private void ConfigurarDGV()
26     {
27         dgvContactos.AutoGenerateColumns = false;
28         dgvContactos.Columns.Clear();
29         dgvContactos.Columns.Add(new DataGridViewTextBoxColumn { HeaderText = "ID", DataPropertyName = "ContactoID", Width = 50 });
30         dgvContactos.Columns.Add(new DataGridViewTextBoxColumn { HeaderText = "Nombre", DataPropertyName = "Nombre", Width = 120 });
31         dgvContactos.Columns.Add(new DataGridViewTextBoxColumn { HeaderText = "Apellido", DataPropertyName = "Apellido", Width = 120 });
32         dgvContactos.Columns.Add(new DataGridViewTextBoxColumn { HeaderText = "No. Contacto", DataPropertyName = "NumContacto", Width = 120 });
33         dgvContactos.Columns.Add(new DataGridViewTextBoxColumn { HeaderText = "Dirección", DataPropertyName = "Direccion", Width = 150 });
34         dgvContactos.Columns.Add(new DataGridViewTextBoxColumn { HeaderText = "Género", DataPropertyName = "Genero", Width = 90 });
35     }
36 }
```

```
36 private void CargarDatos(string filtro = "")
37 {
38     var lista = string.IsNullOrEmpty(filtro)
39         ? _service.ObtenerTodos()
40         : _service.Buscar(filtro);
41
42     dgvContactos.DataSource = null;
43     dgvContactos.DataSource = lista;
44 }
45
46 2 referencias
47 private bool ValidarCampos()
48 {
49     if (string.IsNullOrEmpty(txtNombre.Text)) { MessageBox.Show("Ingrese el nombre."); return false; }
50     if (string.IsNullOrEmpty(txtApellido.Text)) { MessageBox.Show("Ingrese el apellido."); return false; }
51     if (string.IsNullOrEmpty(txtNumContacto.Text)) { MessageBox.Show("Ingrese el número."); return false; }
52     if (cmbGenero.SelectedIndex < 0) { MessageBox.Show("Seleccione el género."); return false; }
53     return true;
54 }
55
56 2 referencias
57 private Contacto ContactoDesdeFormulario(int id = 0)
58 {
59     return new Contacto(
60         id,
61         txtNombre.Text.Trim(),
62         txtApellido.Text.Trim(),
63         txtNumContacto.Text.Trim(),
64         txtDireccion.Text.Trim(),
65         cmbGenero.SelectedItem?.ToString()
66     );
67 }
68
69 4 referencias
70 private void LimpiarFormulario()
71 {
72     txtID.Text = "";
73     txtNombre.Text = "";
74     txtApellido.Text = "";
75     txtNumContacto.Text = "";
76     txtDireccion.Text = "";
77     cmbGenero.SelectedIndex = -1;
78     txtBuscar.Text = "";
79 }
```

6. Validaciones en tiempo real

Se agregaron los eventos SoloLetras_KeyPress y SoloNumeros_KeyPress suscritos a los TextBox correspondientes, impidiendo el ingreso de caracteres inválidos antes de llegar al método de validación.

```
154 //Validaciones en tiempo Real
155 1 referencia
156 private void txtNombre_KeyPress(object sender, KeyPressEventArgs e)
157 {
158     // Permitir letras (incluyendo tildes/ñ), espacios y backspace
159     if (!char.IsLetter(e.KeyChar) && e.KeyChar != ' ' && e.KeyChar != '\b')
160         e.Handled = true;
161 }
162
163 1 referencia
164 private void txtApellido_KeyPress(object sender, KeyPressEventArgs e)
165 {
166     // Permitir letras (incluyendo tildes/ñ), espacios y backspace
167     if (!char.IsLetter(e.KeyChar) && e.KeyChar != ' ' && e.KeyChar != '\b')
168         e.Handled = true;
169 }
170
171 1 referencia
172 private void txtNumContacto_KeyPress(object sender, KeyPressEventArgs e)
173 {
174     // Permitir solo dígitos y backspace
175     if (!char.IsDigit(e.KeyChar) && e.KeyChar != '\b')
176         e.Handled = true;
177 }
```


7. Selección desde el DataGridView

El evento CellClick carga los datos de la fila seleccionada en los controles del formulario, accediendo a las celdas por índice (fila.Cells[0] al [5]) para evitar errores por nombre de columna.

```
141  private void dgvContactos_CellClick(object sender, DataGridViewCellEventArgs e)
142  {
143      if (e.RowIndex < 0) return;
144      var fila = dgvContactos.Rows[e.RowIndex];
145
146      txtID.Text = fila.Cells[0].Value?.ToString();
147      txtNombre.Text = fila.Cells[1].Value?.ToString();
148      txtApellido.Text = fila.Cells[2].Value?.ToString();
149      txtNumContacto.Text = fila.Cells[3].Value?.ToString();
150      txtDireccion.Text = fila.Cells[4].Value?.ToString();
151      cmbGenero.Text = fila.Cells[5].Value?.ToString();
152  }
```

IV. ANALISIS E INTERPRETACION DE RESULTADOS

¿Que indican los resultados?

La aplicación responde correctamente a todas las operaciones CRUD: los contactos se agregan con ID autoincrementado, se actualizan al seleccionarlos del grid, se eliminan con confirmación previa y se limpian del formulario con el botón Claro.

El DataGridView refleja en tiempo real cualquier cambio realizado, ya que se reasigna el DataSource después de cada operación. La búsqueda filtra instantáneamente al escribir en txtBuscar usando el evento TextChanged.

¿Que se ha encontrado?

Se verifico que acceder a las celdas del DataGridView por nombre de columna (HeaderText) genera una excepción System.ArgumentException si el nombre no coincide exactamente. La solución fue acceder por índice numérico, que es más robusto.

Las validaciones con Regex funcionan correctamente: el campo nombre y apellido rechazan números y símbolos, y el número de contacto acepta únicamente dígitos. El doble nivel de validación (KeyPress + Regex) ofrece una experiencia de usuario más fluida y segura.

La separación entre ContactoService y Form1 facilito la identificación y corrección de errores, ya que la lógica de negocio y la interfaz son independientes entre sí.



V. CUESTIONARIO

1. ¿Que es el encapsulamiento y como se aplica en este proyecto?

El encapsulamiento consiste en ocultar los datos internos de una clase y exponerlos solo a través de propiedades o métodos. En Contacto.cs los atributos son propiedades públicas con get/set. En ContactoService la lista _contactos es privada y solo se puede manipular a través de los métodos del servicio.

2. ¿Por qué se usa List<Contacto> en lugar de un arreglo?

Porque List<T> es dinámica: no requiere tamaño fijo, permite agregar y eliminar elementos fácilmente, y ofrece métodos como FirstOrDefault() y Where() mediante LINQ, lo que simplifica las operaciones de búsqueda y filtrado.

3. Que ventaja tiene separar la lógica en ContactoService?

Permite reutilizar la lógica de negocio sin depender de la interfaz gráfica. Si en el futuro se cambia el formulario o se agrega una API, ContactoService no necesita modificarse. Además facilita las pruebas unitarias.

4. ¿Por qué se valida tanto en KeyPress como en ValidarCampos?

El KeyPress previene el ingreso de caracteres inválidos en tiempo real, mejorando la experiencia del usuario. ValidarCampos es una segunda línea de defensa que garantiza que los datos sean correctos antes de procesarlos, por ejemplo, si el valor llega por otra vía.

5. Que produce el error System.ArgumentException al acceder al DataGridView?

Ocurre cuando se intenta acceder a una celda por nombre de columna usando un.HeaderText que no coincide exactamente con el registrado. La solución es acceder por índice numérico (Cells[0], Cells[1], etc.), que es siempre confiable independientemente del texto del encabezado.

CONCLUSIONES

- La aplicación cumple con los principios de POO: la clase Contacto encapsula los datos, ContactoService concentra la lógica, y Form1 solo gestiona la interfaz.
- El uso de List<T> con LINQ simplifica notablemente las operaciones de búsqueda y filtrado sin necesidad de bucles manuales.
- Implementar dos niveles de validación (KeyPress y Regex) resulta en una experiencia más robusta y amigable para el usuario.
- Acceder a las celdas del DataGridView por índice es más seguro que hacerlo por nombre, evitando excepciones en tiempo de ejecución.
- La separación entre lógica de negocio e interfaz facilita el mantenimiento del proyecto y reduce el acoplamiento entre componentes.